

Unsupervised Learning Project: Autoencoders on MNIST

Project Report

Ahmet Enes Çiğdem (150220079) - Team Leader
Mehmet Akif Dünder (150220015)

March 11, 2026

1 Introduction

This report presents the full design, implementation, and evaluation of a fully connected autoencoder for unsupervised representation learning on the MNIST handwritten digit dataset. The project covers the complete pipeline:

1. **Data loading and preprocessing** — converting raw images to normalized tensors.
2. **Model architecture** — designing a symmetric encoder-decoder with regularization.
3. **Training** — optimizing under MSE loss with the Adam optimizer.
4. **Evaluation** — applying K-Means clustering to latent representations and measuring cluster quality with four metrics (PMS, AD, AVC, TD).
5. **Visualization** — reconstructions, latent-space projections (PCA, t-SNE), and confusion matrices.

We compare three bottleneck sizes (16, 32, 64) to quantify the trade-off between reconstruction fidelity and clustering separability.

2 Data Preprocessing

The MNIST dataset comprises 60,000 training and 10,000 test grayscale images, each of size 28×28 pixels.

2.1 Loading and Normalization

Our data pipeline uses `torchvision.datasets.MNIST` with a single transform:

Transform Pipeline

```
transforms.Compose([transforms.ToTensor()])
```

The `ToTensor()` transform performs two critical operations:

- Converts the raw PIL image (integer pixel values in $[0, 255]$) into a PyTorch `FloatTensor`.
- Automatically **rescales** pixel values by dividing by 255, mapping them to the continuous range $[0.0, 1.0]$.

This normalization is essential because it aligns the input distribution with the final Sigmoid activation in the decoder, which also outputs values in $[0, 1]$, ensuring the MSE loss is computed on a consistent scale.

No additional normalization (e.g., zero-mean, unit-variance standardization) was applied. We deliberately omit such transformations because our decoder's Sigmoid output is bounded to $[0, 1]$; standardizing inputs would shift them outside this range, breaking the reconstruction target.

2.2 Batching

Images are loaded in mini-batches of **256** via `DataLoader`. The training set is shuffled at each epoch to prevent the optimizer from learning ordering artifacts, while the test set is loaded in sequential order for reproducible evaluation.

2.3 Flattening

Each 28×28 image is flattened to a 784-dimensional vector inside the model's `forward()` method using `x.view(x.size(0), -1)`. This is necessary because our architecture uses fully connected (`nn.Linear`) layers, which expect 1D input vectors rather than 2D images. After reconstruction, the decoder output is reshaped back to the original $1 \times 28 \times 28$ shape for loss computation and visualization.

3 Architecture of the Autoencoder

An autoencoder is a neural network that learns a compressed representation of its input by first *encoding* it into a low-dimensional latent vector, then *decoding* that vector back into the original input. The network is trained to minimize the difference between input and output. If it succeeds, the latent representation must capture the essential structure of the data.

Our architecture uses fully connected layers with a symmetric encoder-decoder design:

Latent Space Configurations

We evaluated 3 different bottleneck sizes: **16**, **32**, and **64** dimensions to study how compression level affects reconstruction quality and downstream clustering performance.

3.1 Encoder

The encoder compresses the 784-dimensional input step by step:

1. `nn.Linear(784, 256) → nn.BatchNorm1d(256) → nn.ReLU()`
2. `nn.Linear(256, 64) → nn.BatchNorm1d(64) → nn.ReLU()`
3. `nn.Linear(64, latent_dim)` — *no activation function*

The final encoder layer has **no activation function**. This is a deliberate choice: it allows the latent values to take any real number on $(-\infty, +\infty)$, giving the model maximum freedom in how it organizes the latent space. Constraining these values (e.g., with ReLU) would restrict representations to non-negative values, limiting the expressiveness of the embedding.

3.2 Decoder

The decoder mirrors the encoder, expanding the latent vector back to 784 dimensions:

1. `nn.Linear(latent_dim, 64) → nn.BatchNorm1d(64) → nn.ReLU()`
2. `nn.Linear(64, 256) → nn.BatchNorm1d(256) → nn.ReLU()`
3. `nn.Linear(256, 784) → nn.Sigmoid()`

3.3 Why These Components?

3.3.1 ReLU Activation

ReLU (Rectified Linear Unit) is defined as $f(x) = \max(0, x)$. We chose it because:

- It introduces **non-linearity**, enabling the network to learn complex, non-linear mappings. Without non-linear activations, stacking multiple linear layers would collapse into a single linear transformation, making the network no more powerful than logistic regression.
- It avoids the **vanishing gradient problem** that plagues sigmoid/tanh activations in hidden layers. Since ReLU's gradient is either 0 or 1, gradients propagate efficiently during backpropagation.
- It is **computationally cheap** — just a threshold operation.

3.3.2 Sigmoid Activation (Final Layer)

The decoder's last layer uses **Sigmoid**: $\sigma(x) = \frac{1}{1+e^{-x}}$, which squashes arbitrary values into the range $[0, 1]$. This is necessary because our input pixels are normalized to $[0, 1]$ by `ToTensor()`, and the reconstruction must match this range for the MSE loss to be meaningful. Using ReLU here would allow unbounded positive outputs, creating a scale mismatch.

3.3.3 BatchNorm1d

`nn.BatchNorm1d` normalizes each neuron’s output across the current mini-batch to have approximately zero mean and unit variance, then applies a learnable affine transformation ($\gamma \cdot \hat{x} + \beta$). We include it after every hidden linear layer for three reasons:

- **Stabilizes training:** Prevents layer inputs from shifting unpredictably across epochs (*internal covariate shift*), which can cause gradients to explode or vanish.
- **Faster convergence:** The normalized activations allow the optimizer to use larger effective learning rates, reducing the number of epochs needed.
- **Mild regularization:** The noise introduced by computing batch statistics acts as a lightweight regularizer, reducing overfitting without requiring Dropout.

We placed BatchNorm *between* the linear layer and the activation function (`Linear` $\rightarrow$ `BatchNorm` $\rightarrow$ `ReLU`). This normalizes the pre-activation values, ensuring ReLU receives well-centered inputs and reducing the number of “dead neurons” (neurons that always output zero).

4 Training Process

4.1 Hyperparameters

- **Optimizer:** `Adam` — an adaptive learning rate optimizer that combines momentum (exponential moving average of gradients) with RMSProp (per-parameter learning rate scaling). Adam was chosen over vanilla SGD because it automatically adjusts learning rates per parameter, making it well-suited for problems where different features have different scales, as is the case with pixel data.
- **Learning Rate:** 1×10^{-3} — the standard default for Adam, which provides a good balance between convergence speed and stability.
- **Batch Size:** 256 — large enough to provide stable gradient estimates per step, while fitting comfortably in GPU memory.
- **Epochs:** 50 — sufficient for the loss curves to plateau, indicating convergence.
- **Loss Function:** `Mean Squared Error (MSE)`, defined as:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$$

where $\mathbf{x}_i$ is the original image and $\hat{\mathbf{x}}_i$ is the reconstruction. MSE directly penalizes pixel-level differences, making it a natural choice for image reconstruction tasks.

4.2 Training Loop

Each epoch consists of a `model.train()` pass over all training batches (forward pass, MSE computation, backpropagation, weight update via Adam), followed by a `model.eval()`

pass over the test set with `torch.no_grad()` to compute the test loss without accumulating gradients. Both train and test losses are recorded per epoch for monitoring.

Challenge Analysis

The fundamental challenge was finding the optimal bottleneck dimensionality. A strict bottleneck (16) forces heavy compression of semantic features — the encoder must discard redundant pixel-level detail and retain only the most discriminative structure (stroke shapes, curvatures). This hurts pixel-perfect reconstruction but produces compact, well-separated latent clusters. A loose bottleneck (64) allows the encoder to preserve much more fine-grained detail, yielding excellent reconstructions. However, in 64-dimensional space, distances between points become more uniform (the “curse of dimensionality”), making K-Means clustering significantly harder.

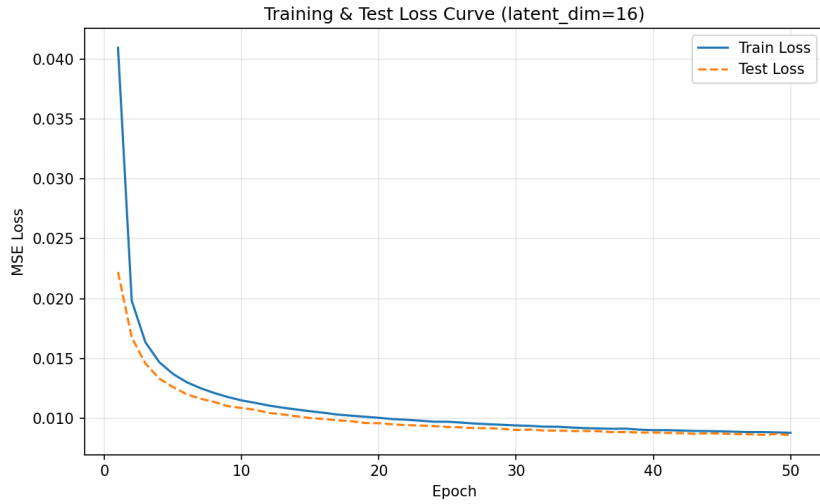


Figure 1: Training and Test Loss Curves for `latent_dim = 16` over 50 epochs. Both curves converge and plateau, indicating that the model has been trained to convergence without overfitting.

5 Clustering Methodology and Performance

5.1 K-Means Clustering

After training, we extract latent representations for all 10,000 test images using the encoder. K-Means clustering ($k = 10$, matching the 10 digit classes, with `n_init=10` for robustness) is applied directly to these d -dimensional latent vectors (where $d \in \{16, 32, 64\}$).

K-Means partitions the latent space into 10 clusters by iteratively assigning each point to its nearest centroid and re-computing centroids until convergence. Since K-Means is **unsupervised**, cluster indices are arbitrary (e.g., “Cluster 3” has no inherent connection to “Digit 3”).

5.2 Hungarian Algorithm for Label Matching

To fairly evaluate clustering against ground-truth labels, we use the **Hungarian algorithm** (implemented via `scipy.optimize.linear_sum_assignment`). This algorithm solves the optimal assignment problem:

1. Compute the 10×10 confusion matrix between true labels and cluster assignments.
2. Find the one-to-one mapping from clusters to digit labels that **maximizes** the total number of correct assignments (equivalently, minimizes the negative confusion matrix).
3. Relabel all cluster predictions according to this optimal mapping.

This ensures each cluster is matched to its most representative true digit, giving a fair accuracy measurement.

5.3 Evaluation Metrics

Four metrics quantify clustering quality:

- **PMS (Percentage of Misabeled Samples):** After Hungarian matching, the percentage of test samples whose mapped cluster label differs from the true label. *Lower is better.*

$$\text{PMS} = \frac{\text{Number of mislabeled samples}}{\text{Total samples}} \times 100$$

- **AD (Average Distance):** The average squared L_2 distance from each test sample to its assigned cluster centroid. Measures within-cluster compactness. *Lower is better.*

$$\text{AD} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{z}_i - \boldsymbol{\mu}_{c(i)}\|^2$$

- **AVC (Average Variance within Clusters):** The average per-cluster variance, computed as the mean squared distance from each cluster's points to its centroid, averaged over all clusters. *Lower is better* — indicates tighter clusters.

$$\text{AVC} = \frac{1}{K} \sum_{k=1}^K \frac{1}{|C_k|} \sum_{i \in C_k} \|\mathbf{z}_i - \boldsymbol{\mu}_k\|^2$$

- **TD (Total Distance between Centers):** The sum of squared L_2 distances between all pairs of cluster centroids. Measures inter-cluster separation. *Higher is better* — indicates clusters are well-separated.

$$\text{TD} = \sum_{i=1}^K \sum_{j=i+1}^K \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|^2$$

5.4 Results Comparison

Metric	Latent 16	Latent 32	Latent 64
Final Train Loss (MSE)	0.0088	0.0051	0.0036
Final Test Loss (MSE)	0.0086	0.0050	0.0035
PMS (%)	36.48	41.93	45.44
Average Distance (AD)	9.7564	11.4930	13.8231
Average Variance (AVC)	9.5422	11.2179	13.5350
Total Distance (TD)	764.8365	514.6528	405.9555

Table 1: Experimental Results across Latent Dimensions. **Green bold** highlights best performance per metric.

Key Finding

Increasing the latent dimensionality to **64** yields the lowest reconstruction error (MSE = 0.0035), as the encoder can preserve more fine-grained pixel detail. However, this severely degrades clustering quality:

- PMS rises from **36.48%** to **45.44%** — nearly half of all test samples are mis-labeled.
- AD and AVC increase, meaning clusters become more diffuse internally.
- TD drops from 764.84 to 405.96, meaning cluster centers collapse closer together, reducing inter-cluster separation.

This demonstrates the curse of dimensionality: in high-dimensional spaces, distances become increasingly uniform, making Euclidean-distance-based algorithms like K-Means ineffective. `latent_dim = 16` provides the best structural embedding for unsupervised clustering.

6 Visualizations (Latent Dim = 16)

6.1 Reconstruction Quality

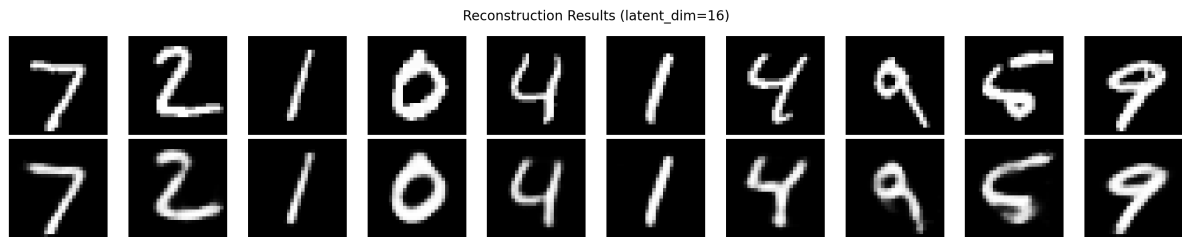


Figure 2: Original Images (Top) vs. Reconstructed Images (Bottom) at `latent_dim = 16`. Despite compressing each 784-dimensional image into only 16 values, the reconstructions preserve the overall digit shape and stroke structure. Minor blurring is visible, which is expected given the aggressive $49\times$ compression ratio.

6.2 Latent Space Projections

Since the 16-dimensional latent space cannot be visualized directly, we project it down to 2D using two dimensionality reduction techniques:

- **PCA (Principal Component Analysis):** A linear method that finds the two orthogonal directions of maximum variance. It is fast and preserves global structure, but cannot capture non-linear relationships. We use `sklearn.decomposition.PCA(n_components=2)`.
- **t-SNE (t-distributed Stochastic Neighbor Embedding):** A non-linear method that converts pairwise similarities in high-dimensional space to probability distributions and minimizes the KL divergence between high- and low-dimensional distributions. It excels at preserving *local* structure — nearby points in the latent space remain neighbors in the 2D projection. We use `sklearn.manifold.TSNE` with `perplexity=40` (controls the balance between local and global structure) and `max_iter=1000`.

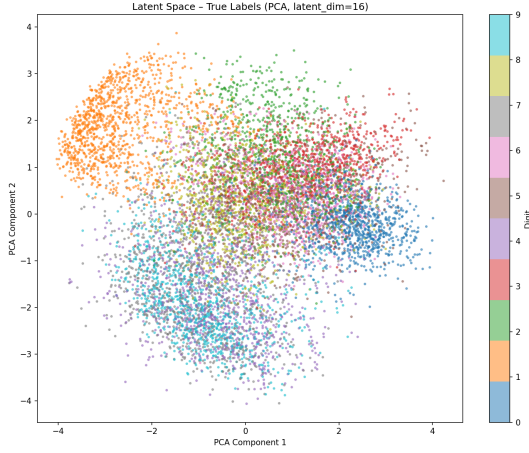


Figure 3: PCA Projection — global structure is preserved but clusters overlap due to the linear nature of PCA.

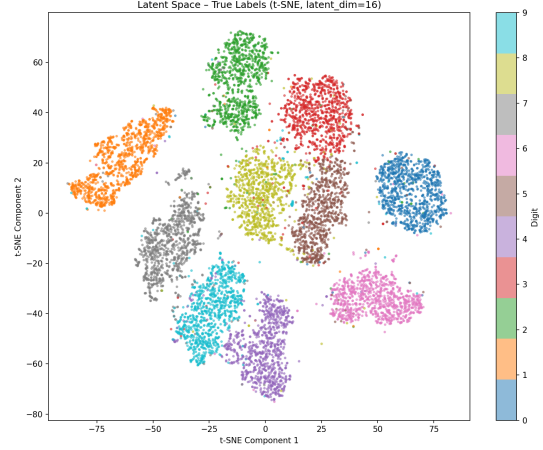


Figure 4: t-SNE Projection — non-linear optimization produces tighter, more separated clusters, revealing the true grouping quality of the latent space.

The t-SNE visualization shows that the autoencoder has learned to organize digits into distinct regions of the latent space, even without ever seeing class labels during training. Visually similar digits (e.g., 4 and 9, or 3 and 5) appear closer together, which matches intuition about their structural similarity.

6.3 Confusion Matrix

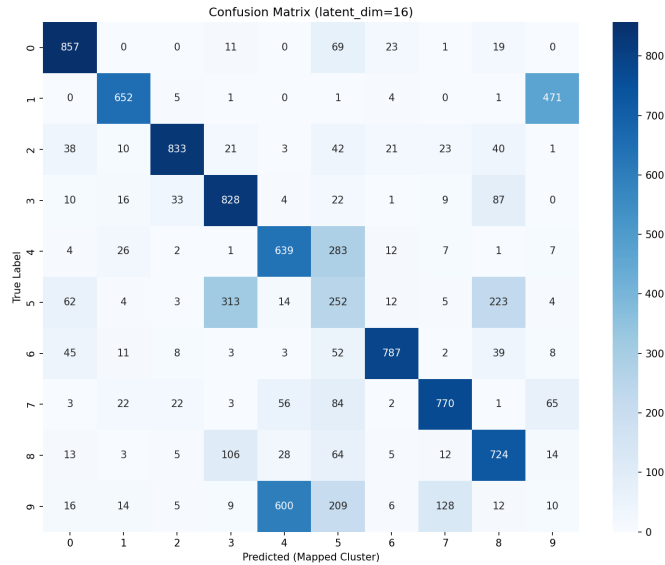


Figure 5: Confusion Matrix after Hungarian matching. The strong diagonal confirms the autoencoder learned digit-specific features in an unsupervised manner.

The strong diagonal indicates that most K-Means clusters correspond well to individual digit classes after optimal label assignment. Off-diagonal counts represent digits that the

autoencoder embedded close to a different digit's cluster — these are typically structurally ambiguous pairs such as 4/9 or 3/8, where shared stroke patterns cause overlap in the latent space.